

# Investigação dos Efeitos do Paralelismo ao Nível de Instrução (ILP): Pipeline, Dependências e Vetorização em C

Israel Soares de Castro Filho  
Matrícula: 20250070780

## 1 Introdução

Processadores modernos utilizam *pipelining* e múltiplas unidades funcionais para executar instruções de forma sobreposta, explorando o chamado paralelismo ao nível de instrução (*Instruction-Level Parallelism* – ILP). No entanto, esse paralelismo só pode ser aproveitado quando não há dependências verdadeiras (*read-after-write*) entre instruções consecutivas: quando uma instrução precisa do resultado da anterior para ser executada, o processador é forçado a esperar, criando bolhas no pipeline e reduzindo o throughput de instruções por ciclo (IPC).

O objetivo deste trabalho é investigar experimentalmente esses efeitos por meio de três laços escritos em linguagem C: (1) um laço de inicialização de vetor, cujas iterações são independentes entre si; (2) um laço de soma acumulativa com uma única variável, que cria uma cadeia de dependência serial entre iterações; e (3) uma versão da soma que quebra essa dependência distribuindo o acúmulo em quatro variáveis independentes. Os três laços foram compilados com diferentes níveis de otimização do GCC (-O0, -O2 e -O3), e os tempos de execução foram comparados para avaliar como o estilo do código-fonte e a presença de dependências entre iterações influenciam o desempenho final, tanto na ausência quanto na presença de otimizações automáticas do compilador.

## 2 Metodologia

Foi implementado um programa em C (Código 1, no Apêndice A) contendo três funções:

- **Laço 1 – Inicialização do vetor:** preenche um vetor de  $N = 10^8$  elementos `double` com a expressão `v[i] = i * 0.5 + 1.0`. Cada elemento depende apenas do índice `i`, portanto não há dependência entre iterações.
- **Laço 2 – Soma dependente:** acumula os elementos do vetor em uma única variável (`soma += v[i]`), criando uma cadeia de dependência estritamente serial: a adição da iteração  $i$  só pode começar depois que o resultado da iteração  $i - 1$  estiver disponível.
- **Laço 3 – Soma independente (múltiplos acumuladores):** distribui a soma em quatro variáveis (`s0`, `s1`, `s2`, `s3`), processando os elementos em grupos de quatro. Cada uma das quatro cadeias ainda é serial internamente, porém as quatro cadeias

são independentes entre si e podem, em princípio, ser executadas em paralelo pelo processador ou vetorizadas pelo compilador.

O tempo de cada laço foi medido com `clock_gettime(CLOCK_MONOTONIC, ...)`, tomando a média de 5 repetições por execução, a fim de reduzir a influência de ruído do sistema operacional. O programa foi compilado três vezes com o GCC, variando apenas o nível de otimização:

- `gcc -O0 -o ilp_00 ilp_pipeline.c`
- `gcc -O2 -o ilp_02 ilp_pipeline.c`
- `gcc -O3 -o ilp_03 ilp_pipeline.c`

Cada um dos três binários foi executado 3 vezes, e os tempos médios de cada laço, junto com o *speedup* do Laço 3 em relação ao Laço 2, foram registrados para análise.

### 3 Resultados

A Tabela 1 apresenta os tempos brutos (em segundos) obtidos nas três execuções de cada binário, e a Tabela 2 resume as médias por nível de otimização.

Tabela 1: Tempos de execução (em segundos) das três execuções de cada binário

| Otimização | Execução | Laço 1 | Laço 2 | Laço 3 |
|------------|----------|--------|--------|--------|
| -O0        | 1        | 0.0977 | 0.1654 | 0.0574 |
|            | 2        | 0.1058 | 0.1655 | 0.0589 |
|            | 3        | 0.0891 | 0.1629 | 0.0580 |
| -O2        | 1        | 0.0558 | 0.0526 | 0.0369 |
|            | 2        | 0.0561 | 0.0524 | 0.0353 |
|            | 3        | 0.0565 | 0.0515 | 0.0363 |
| -O3        | 1        | 0.0571 | 0.0515 | 0.0355 |
|            | 2        | 0.0567 | 0.0516 | 0.0361 |
|            | 3        | 0.0566 | 0.0535 | 0.0364 |

Tabela 2: Tempos médios (s) e speedup do Laço 3 em relação ao Laço 2

| Otimização | Laço 1 (s) | Laço 2 (s) | Laço 3 (s) | Speedup (L2/L3) |
|------------|------------|------------|------------|-----------------|
| -O0        | 0.0975     | 0.1646     | 0.0581     | 2.83×           |
| -O2        | 0.0561     | 0.0522     | 0.0362     | 1.44×           |
| -O3        | 0.0568     | 0.0522     | 0.0360     | 1.45×           |

Adicionalmente, comparando o ganho absoluto proporcionado pela otimização do compilador (independentemente do estilo do código), tem-se, saindo de -O0 para -O2:

- Laço 1 (inicialização): speedup de aproximadamente 1.74×

- Laço 2 (soma dependente): speedup de aproximadamente  $3.15\times$ ;
- Laço 3 (múltiplos acumuladores): speedup de aproximadamente  $1.60\times$ .

Já entre -02 e -03, os tempos ficaram praticamente idênticos (diferenças inferiores a 1% em todos os laços), indicando que, para este código relativamente simples, o -02 já ativa a maior parte das otimizações relevantes.

## 4 Análise

### 4.1 Laço 1 – Ausência de dependência

No Laço 1, cada elemento do vetor é calculado exclusivamente a partir do índice  $i$ , sem depender de nenhum resultado anterior. Essa característica torna o laço trivialmente paralelizável: o compilador pode, a partir de -02, aplicar vetorização automática (SIMD), processando vários elementos do vetor em uma única instrução, além de reordenar e sobrepor instruções livremente no pipeline. Isso explica o ganho expressivo de desempenho ( $1.74\times$ ) ao sair de -00 para -02.

### 4.2 Laço 2 – Dependência serial forte

O Laço 2 apresenta uma dependência de dados verdadeira (RAW) entre iterações consecutivas: o valor de `soma` após a iteração  $i$  é necessário para calcular a iteração  $i + 1$ . Essa cadeia serial impede que as instruções de adição sejam executadas fora de ordem ou sobrepostas de forma significativa, mesmo que o processador possua múltiplas unidades aritméticas livres – o gargalo não é a disponibilidade de hardware, mas a *latência* da cadeia de dependência.

Curiosamente, esse foi o laço que mais se beneficiou da otimização do compilador em termos relativos ( $3.15\times$  de -00 para -02). Isso ocorre porque em -00 o compilador gera código extremamente literal, sem sequer manter a variável acumuladora em registrador entre iterações, gerando muitas operações redundantes de leitura/escrita em memória. Já em -02/-03, o GCC aplica otimizações como *register allocation* eficiente e, sob a flag `-ffast-math` (não usada aqui) poderia até reassociar a soma; mesmo sem essa flag, o compilador reduz drasticamente o overhead de acesso à memória, mas a dependência estrutural entre as somas permanece, o que explica por que o Laço 2 continua sendo o mais lento dos três mesmo nos níveis mais altos de otimização.

### 4.3 Laço 3 – Quebra da dependência com múltiplos acumuladores

Ao dividir a soma em quatro variáveis independentes (`s0`, `s1`, `s2`, `s3`), o Laço 3 substitui uma única cadeia de dependência longa por quatro cadeias mais curtas e independentes entre si. Isso permite que o processador execute as quatro somas em paralelo, aproveitando melhor as unidades funcionais de ponto flutuante disponíveis e escondendo a latência de cada adição individual atrás da execução das demais.

O resultado é consistente em todos os níveis de otimização: o Laço 3 é sempre mais rápido que o Laço 2, com speedups de  $2.83\times$  (-00),  $1.44\times$  (-02) e  $1.45\times$  (-03). É interessante notar que a vantagem relativa do Laço 3 é *maior* em -00: sem nenhuma otimização automática, a estrutura do código-fonte (quantas cadeias de dependência independentes existem) é o único fator que determina o quanto de ILP pode ser explorado, já que o

compilador não reescreve o código. Em -02/-03, parte do ganho que o programador obteria manualmente já é parcialmente alcançada pelo compilador ao otimizar o Laço 2 (por exemplo, mantendo o acumulador em registrador e otimizando o acesso à memória), o que reduz — mas não elimina — a vantagem relativa do Laço 3.

## 4.4 O que cada nível de otimização faz

- **-O0 (sem otimização):** o GCC gera código de forma direta e literal, sem reordenar instruções, sem manter variáveis em registradores entre iterações do laço quando isso não é trivial, e sem aplicar vetorização. É o nível que melhor expõe o comportamento “cru” do código-fonte, e por isso é o que mais penaliza o laço com dependência forte (Laço 2) e mais valoriza a estrutura de múltiplos acumuladores (Laço 3).
- **-O2 (otimização padrão para produção):** ativa um conjunto amplo de otimizações, como alocação eficiente de registradores, eliminação de código morto, *inlining* de funções pequenas, desenrolamento de laços (*loop unrolling*) e vetorização automática (auto-vectorization) quando não há dependências que a impeçam. É nesse nível que se observa o maior salto de desempenho em relação a -O0, especialmente no Laço 2, cuja dependência de dados limita — mas não impede — os ganhos de eficiência na geração de código.
- **-O3 (otimização agressiva):** inclui todas as otimizações de -O2 e adiciona transformações mais agressivas, como vetorização mais ampla, melhor desenrolamento de laços e otimizações de *tree vectorization*. Neste experimento, porém, os tempos entre -O2 e -O3 foram praticamente idênticos, o que sugere que, para laços tão simples e regulares como estes, -O2 já captura a maior parte dos benefícios de vetorização e ILP possíveis, e as otimizações adicionais de -O3 não encontram mais oportunidades relevantes de melhoria neste código específico.

## 5 Conclusão

Os resultados confirmam experimentalmente que dependências verdadeiras entre iterações de um laço (Laço 2) limitam severamente a exploração do paralelismo ao nível de instrução, mesmo em processadores com múltiplas unidades funcionais e mesmo sob otimização agressiva do compilador. Por outro lado, reestruturar o código para quebrar essa dependência em múltiplas cadeias independentes (Laço 3) permite ao hardware — e ao compilador, quando aplicável — explorar esse paralelismo de forma mais efetiva, resultando em speedups consistentes em todos os níveis de otimização testados.

Além disso, observou-se que grande parte do ganho de desempenho de -O0 para -O2 vem da geração de código mais eficiente pelo compilador (uso de registradores, eliminação de acessos redundantes à memória, vetorização automática), mas que essas otimizações automáticas não conseguem eliminar completamente o custo de uma dependência de dados verdadeira imposta pela lógica do algoritmo. Isso reforça a importância de o programador estar ciente de como o estilo do código-fonte — em particular, a presença ou ausência de dependências entre iterações — impacta diretamente a capacidade do processador (e do compilador) de explorar o paralelismo ao nível de instrução, independentemente do nível de otimização utilizado na compilação.

# A Código-Fonte

Código utilizado para gerar os resultados apresentados neste relatório.

```
1  /*
2  * Compile com diferentes níveis de otimização e compare os tempos:
3  *
4  * gcc -O0 -o ilp_00 ilp_pipeline.c
5  * gcc -O2 -o ilp_02 ilp_pipeline.c
6  * gcc -O3 -o ilp_03 ilp_pipeline.c
7  *
8  * Execute várias vezes cada binário para obter uma média confiável:
9  *
10 * ./ilp_00
11 * ./ilp_02
12 * ./ilp_03
13 */
14
15 #include <stdio.h>
16 #include <stdlib.h>
17 #include <time.h>
18
19 #define N 100000000 /* tamanho do vetor (100 milhões de elementos) */
20 #define REPS 5
21
22 double medir_tempo(struct timespec inicio, struct timespec fim) {
23     return (fim.tv_sec - inicio.tv_sec) +
24         (fim.tv_nsec - inicio.tv_nsec) / 1e9;
25 }
26
27 // Laço 1: inicialização do vetor
28
29 void inicializa_vetor(double *v, int n) {
30     for (int i = 0; i < n; i++) {
31         v[i] = (double)i * 0.5 + 1.0;
32     }
33 }
34
35 // Laço 2: soma acumulativa com dependência (uma única variável)
36
37 double soma_dependente(double *v, int n) {
38     double soma = 0.0;
39     for (int i = 0; i < n; i++) {
40         soma += v[i]; /* soma[i] depende de soma[i-1] */
41     }
42     return soma;
43 }
44
45 // Laço 3: soma com múltiplos acumuladores (quebra a dependência)
46
47 double soma_independente(double *v, int n) {
48     double s0 = 0.0, s1 = 0.0, s2 = 0.0, s3 = 0.0;
49     int i;
50
51     /* processa de 4 em 4 elementos */
52     for (i = 0; i + 3 < n; i += 4) {
53         s0 += v[i];
54         s1 += v[i + 1];
```

```

55     s2 += v[i + 2];
56     s3 += v[i + 3];
57 }
58
59 /* trata elementos restantes, caso n não seja múltiplo de 4 */
60 for (; i < n; i++) {
61     s0 += v[i];
62 }
63
64 return (s0 + s1) + (s2 + s3);
65 }
66
67 int main(void) {
68     double *v = malloc(N * sizeof(double));
69     if (!v) {
70         fprintf(stderr, "Erro ao alocar memoria\n");
71         return 1;
72     }
73
74     struct timespec t0, t1;
75     double tempo_init = 0, tempo_dep = 0, tempo_indep = 0;
76     double resultado_dep = 0, resultado_indep = 0;
77
78     printf("Vetor de %d elementos, %d repeticoes por laço\n", N, REPS);
79
80     /* ---- Laço 1: inicialização ---- */
81     for (int r = 0; r < REPS; r++) {
82         clock_gettime(CLOCK_MONOTONIC, &t0);
83         inicializa_vetor(v, N);
84         clock_gettime(CLOCK_MONOTONIC, &t1);
85         tempo_init += medir_tempo(t0, t1);
86     }
87     tempo_init /= REPS;
88
89     /* ---- Laço 2: soma dependente ---- */
90     for (int r = 0; r < REPS; r++) {
91         clock_gettime(CLOCK_MONOTONIC, &t0);
92         resultado_dep = soma_dependente(v, N);
93         clock_gettime(CLOCK_MONOTONIC, &t1);
94         tempo_dep += medir_tempo(t0, t1);
95     }
96     tempo_dep /= REPS;
97
98     /* ---- Laço 3: soma independente (múltiplos acumuladores) ---- */
99     for (int r = 0; r < REPS; r++) {
100         clock_gettime(CLOCK_MONOTONIC, &t0);
101         resultado_indep = soma_independente(v, N);
102         clock_gettime(CLOCK_MONOTONIC, &t1);
103         tempo_indep += medir_tempo(t0, t1);
104     }
105     tempo_indep /= REPS;
106
107     printf("Laco 1 (inicializacao).....: %.4f s\n", tempo_init);
108     printf("Laco 2 (soma dependente).....: %.4f s (resultado = %.2f)\n",
109           tempo_dep, resultado_dep);
110     printf("Laco 3 (soma c/ 4 acumuladores): %.4f s (resultado = %.2f)\n",
111           tempo_indep, resultado_indep);
112

```

```
113     printf("\nSpeedup do laco 3 em relacao ao laco 2: %.2fx\n",
114           tempo_dep / tempo_indep);
115
116     free(v);
117     return 0;
118 }
```

Código 1: Programa em C utilizado para o experimento de ILP.